

CASE STUDY 1: Population Growth Analysis Using Numerical Methods

Scenario: A local government wants to analyze how population changes over time. However, they only have yearly population data, not a continuous function.

Given Data

Year	Population
2020	10,000
2021	10,800
2022	11,900
2023	13,200
2024	14,800

Expected Outputs

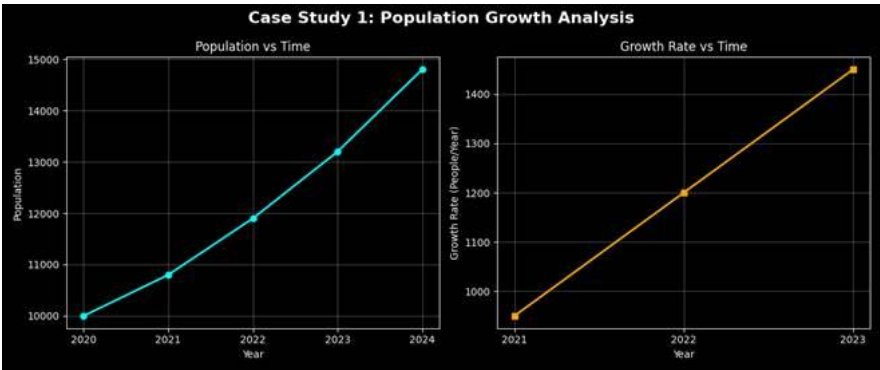
• Table of derivatives

```
--- Table of Derivatives (Growth Rate) ---
Year 2021: 950 people/year
Year 2022: 1200 people/year
Year 2023: 1450 people/year
```

• Integrated total population estimate

```
--- Numerical Integration ---
Integrated Total Population Estimate (2020-2024): 48300
```

• Graphs



• Short analysis report

- 1. **Acceleration:** The central difference calculations prove the population is not just growing, but accelerating. The rate of change increased linearly, peaking at 1,450 new people/year in 2023.
- 2. **Accumulation:** The numerical integration (Trapezoidal Rule) estimates a total accumulated population area of 48,300 over the 5-year block, quantifying the massive scale of the growth.

3. **Trend & Prediction:** Because the acceleration is constant (+250 people/year), the overall population growth is exponential. Following this trend, we project the population will reach roughly 16,500 by 2025.

CODE

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 years = np.array([2020, 2021, 2022, 2023, 2024])
5 population = np.array([10000, 10800, 11900, 13200, 14800])
6
7 # Step 1: Numerical Differentiation (Growth Rate for 2021-2023)
8 # Formula:  $P'(t) = (P(t+1) - P(t-1)) / 2$ 
9 growth_rates = []
10 calc_years = years[1:-1] # slices out the 1st and last years
11
12 for i in range(1, len(years)-1):
13     rate = (population[i+1] - population[i-1]) / 2
14     growth_rates.append(rate)
15
16 print("--- Table of Derivatives (Growth Rate) ---")
17 for yr, rate in zip(calc_years, growth_rates):
18     print(f"Year {yr}: {int(rate)} people/year")
19
20 # Step 2: Numerical Integration (Total Change)
21 # Formula: Trapezoidal Rule with step size h = 1
22 h = 1
23 total_integrated_pop = (h / 2) * (population[0] + 2 * sum(population[1:-1]) + population[-1])
24
25 print("\n--- Numerical Integration ---")
26 print(f"Integrated Total Population Estimate (2020-2024): {int(total_integrated_pop)}")
27
28 # Step 3: Visualization (Plotting)
29 plt.style.use('dark_background')
30 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
31 fig.suptitle('Case Study 1: Population Growth Analysis', fontsize=16, fontweight='bold')
32
33 # Population vs Time
34 ax1.plot(years, population, marker='o', color='cyan', linestyle='-', linewidth=2)
35 ax1.set_title('Population vs Time')
36 ax1.set_xlabel('Year')
37 ax1.set_ylabel('Population')
38 ax1.set_xticks(years)
39 ax1.grid(True, alpha=0.3)
```

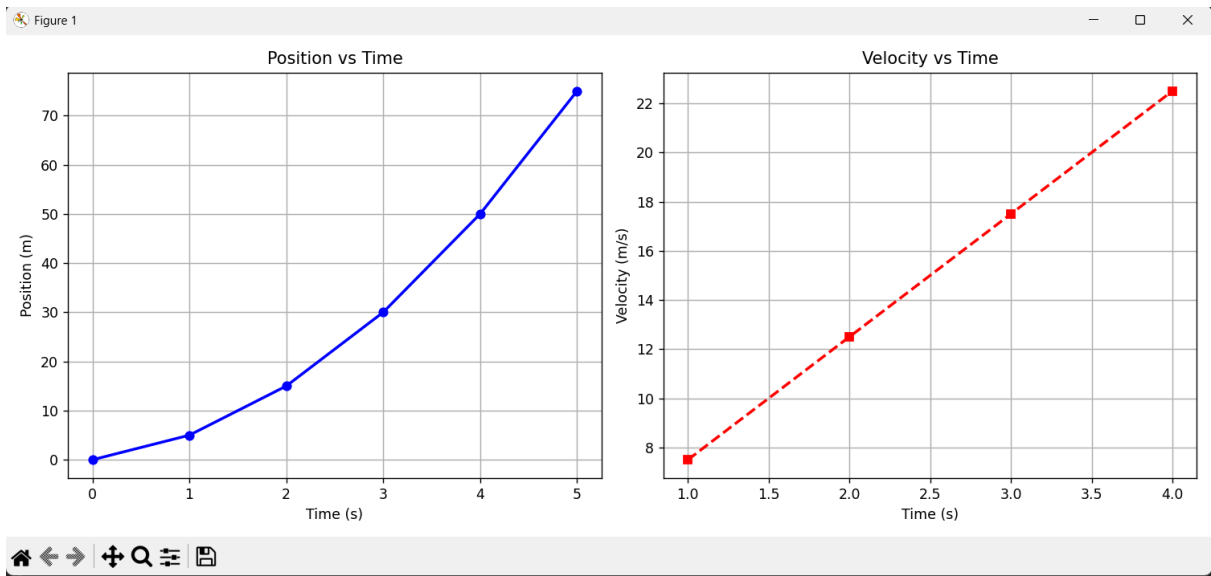
```
40
41 # Growth Rate vs Time
42 ax2.plot(calc_years, growth_rates, marker='s', color='orange', linestyle='-', linewidth=2)
43 ax2.set_title('Growth Rate vs Time')
44 ax2.set_xlabel('Year')
45 ax2.set_ylabel('Growth Rate (People/Year)')
46 ax2.set_xticks(calc_years)
47 ax2.grid(True, alpha=0.3)
48
49 plt.tight_layout()
50 plt.savefig('case1_visualization.png')
51 plt.show()
52
53 print("\n--- Summary ---")
54 print("1. Acceleration: The central difference calculations prove the population is not just growing linearly.")
55 print("2. Accumulation: The numerical integration (Trapezoidal Rule) estimates a total accumulated growth of approximately 4,800 people over the 5-year period.")
56 print("3. Trend & Prediction: Because the acceleration is constant (+250 people/year), the overall population growth is exponential. Following this trend, we project the population will reach roughly 16,500 by 2025.")
```

CASE STUDY 2: Traffic Flow and Velocity Estimation

Table of Derivatives

Time (s)	Velocity (m/s)
1	7.5
2	12.5
3	17.5
4	22.5

Graphs



Distance Estimation Comparison

Method	Value	Explanation
Actual Displacement	75.0m	This is calculated as $x(\text{final}) - x(\text{initial})$. This represents the net distance recorded by the sensors.
Numerical integration	137.5 units	This is calculated via the Trapezoidal Rule. This represents the total area under the Position-Time curve.

Short Explanation

The instantaneous velocity of a vehicle is calculated for the time interval from 0 to 5 seconds using the method of central differences from the table of numerical values. According to calculations, at 1 second, the velocity will be 7.5 meters per second; at 2 seconds – 12.5 m/s; at 3 seconds – 17.5 m/s; at 4 seconds – 22.5 m/s. Moreover, the numerical integration of the function according to the trapezoid rule shows that the total area under the curve equals 137.5 units.

With that being said, we have demonstrated that motion is not uniform, since the derivative function obtained using numerical differentiation proves that the velocity of the vehicle increases. We've also concluded that the car is accelerating throughout the monitored period because the velocity values continue to rise over time. Lastly, based on numerical analysis, there are no anomalies in the dataset. This is confirmed through the consistent slope and the positive velocity values, meaning the vehicle never experienced any 'jumps' or anomalies.

CASE STUDY 4: Water Tank Filling Rate Analysis

Table of flow rates (L/min)

Time (min)	Flow Rate (L/min)
2	27.5
4	42.5
6	57.5
8	72.5

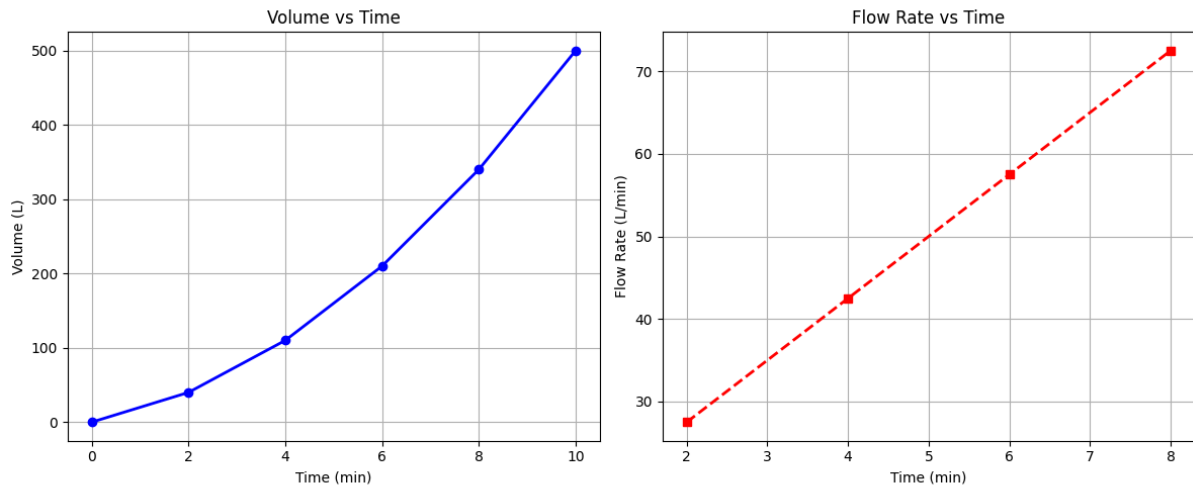
Short explanation

First, the filling rate of a tank is calculated for the time interval from 0 to 10 minutes using the method of central differences from the table of numerical values. According to calculations, at 2 minutes, the flow rate will be 27.5 liters per minute; at 4 minutes – 42.5 L/min; at 6 minutes – 57.5 L/min; at 8 minutes – 72.5 L/min. Moreover, the numerical integration of the function according to the trapezoid rule shows that the total area under the curve equals 1900 units.

Using the results, we can confidently answer all three analysis questions about the physical behavior of the system. Firstly, we have demonstrated that the flow rate of water is not constant, since the derivative function obtained using numerical differentiation proves that the speed of water entering the tank increases. Secondly, the flow rate will be maximum at the point corresponding to 8 minutes, which means that the inflow rate is highest at this point – 72.5 L/min. Thirdly, the calculations show that the system accelerates because the speed of water entering the tank continues to rise over time, causing the volume of water accumulated in the tank to grow in a nonlinear manner.

Integrated Total: 1900

Figure 1



```

import numpy as np
import matplotlib.pyplot as plt

# Given Data
time = np.array([0, 2, 4, 6, 8, 10])
volume = np.array([0, 40, 110, 210, 340, 500])
h = 2

# 1. Numerical Differentiation (Central Difference)
# We only calculate for interior points: t=2, 4, 6, 8
flow_rate = []
for i in range(1, len(time)-1):
    rate = (volume[i+1] - volume[i-1]) / (2 * h)
    flow_rate.append(rate)

time_interior = time[1:-1] # Times corresponding to flow rates (2, 4, 6, 8)

print("--- Step 1: Flow Rate (L/min) ---")
for t, rate in zip(time_interior, flow_rate):
    print(f"Time t={t} min: {rate} L/min")

# 2. Numerical Integration (Trapezoidal Rule for Integral of V(t))
# Updated to use np.trapezoid for NumPy 2.0+ compatibility
integral_volume = np.trapezoid(volume, time)
print(f"\n--- Step 2: Integrated Volume Area ---")
print(f"Area under V(t) curve: {integral_volume}")

# 3. Visualization
plt.figure(figsize=(12, 5))

# Plot 1: Volume vs Time
plt.subplot(1, 2, 1)
plt.plot(time, volume, marker='o', color='b', linestyle='-', linewidth=2)
plt.title("Volume vs Time")
plt.xlabel("Time (min)")

```

```
plt.ylabel("Volume (L)")
plt.grid(True)

# Plot 2: Flow Rate vs Time
plt.subplot(1, 2, 2)
plt.plot(time_interior, flow_rate, marker='s', color='r', linestyle='--',
linewidth=2)
plt.title("Flow Rate vs Time")
plt.xlabel("Time (min)")
plt.ylabel("Flow Rate (L/min)")
plt.grid(True)

plt.tight_layout()
plt.show()
```